

PLATFORM DISCOVERY DOCUMENT

Technology Stack & System Interconnection

A complete technical discovery of the LoadKaro freight marketplace — every runtime surface, the libraries each one is built from, and the exact contracts that bind them to a single Supabase backend.

DOCUMENT DATE

11 August 2026

REPOSITORY

mobile App (monorepo)

REVISION

1.0

00 Contents

01	Platform at a Glance	09	Dashboard API Surface
02	Surface Inventory	10	Security & Authorization
03	Technology Stack — Full Inventory	11	Storage Subsystem
04	System Topology	12	Configuration & Environments
05	Connection Contracts	13	Build, Release & Deployment
06	Identity & Authentication	14	Repository Layout
07	Data Model	15	Architectural Observations
08	Core Data Flows	16	Sources & Verification

How to read this document

Sections 1–4 establish *what exists*. Sections 5–11 establish *how the pieces talk to each other* — the wire-level contracts, keys, and trust boundaries. Sections 12–14 cover operational reality (config, build, repo). Section 15 records architectural observations found during discovery; section 16 states exactly which facts were verified against live code versus inherited from in-repo documentation.

SCOPE NOTE

This is a discovery document, not a design proposal. It describes the platform as it exists in the repository on the document date. Where in-repo documentation could not be re-verified against a live system, that is called out explicitly rather than presented as fact.

01 Platform at a Glance

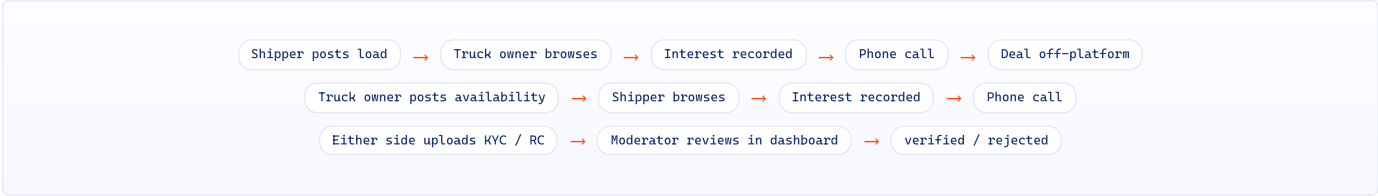
LoadKaro is a **two-sided freight marketplace for the Indian road-logistics market**. Shippers post loads that need moving; truck owners post the availability of their vehicles. The platform surfaces each side to the other, records expressions of interest, and hands off to a phone call for the actual negotiation. A moderation layer verifies the identity of users and the documentation of vehicles.

BUSINESS MODEL Marketplace / discovery — not a 3PL	MATCHING MECHANISM Browse → call (tel:) + interest record
RUNTIME SURFACES 3 (mobile, dashboard, backend)	BACKEND Single Supabase project
PRIMARY AUTH Phone OTP (mobile) · Email+password (staff)	APPLICATION TABLES 11 in public schema
USER ROLES 5 (shipper , truck_owner , broker , moderator , admin)	LANGUAGES SHIPPED 4 (EN, हिन्दी, ಕನ್ನಡ, తెలుగు)

The five defining architectural decisions

DECISION	WHAT IT MEANS	CONSEQUENCE
Single shared backend	Both client applications point at one Supabase project — one Postgres database, one auth provider, one storage bucket.	No service-to-service API layer exists between the apps. The database schema is the integration contract.
Two different trust postures	Mobile uses the anon key and is fully governed by Row Level Security. The dashboard uses the service_role key on server-only routes and bypasses RLS entirely.	Mobile security lives in the database; dashboard security lives in application code. Two separate enforcement systems must stay in agreement.
Phone-first identity	Marketplace users authenticate only by SMS OTP. Email/password is reserved for staff.	Removes password friction for the target user, but couples user acquisition to SMS deliverability and Indian DLT registration.
No realtime, no push	All discovery is pull-based: the user opens a list screen and it queries. No WebSocket subscriptions, no push notifications.	Simple and cheap. Freshness depends on the user re-opening a screen; there is no server-initiated re-engagement channel.
Call-based matching	The platform does not broker, price, or escrow. It opens a tel: link and writes an interests row.	Minimal liability and regulatory surface; the interests table is the only telemetry available for measuring marketplace liquidity.

Value chain in one line



02 Surface Inventory

The platform has exactly three runtime surfaces. Everything else is a library, a config file, or a build artifact.

Surface	Runtime	Deploy Target	Audience	Purpose
LoadKaro mobile LoadKaro/	React Native 0.81.5 on Expo SDK 54, Hermes engine	Android APK / AAB via EAS Build (package in .loadkaro.app); iOS target configured but Android-first	Shippers, truck owners, brokers	The marketplace itself — post, browse, call, express interest, upload verification documents
Admin dashboard admin-dashboard/	Next.js 16 App Router on Node, React 19	Node server (next build → next start)	Internal admins and moderators	Back-office — verification review, record CRUD, KPIs, lead tracking, audit trail
Supabase backend project ffbbujudebmtbuyssnsn	Postgres 17.6 + GoTrue auth + Storage, region ap-south-1 (Mumbai)	Supabase managed cloud	Both applications	Persistence, identity, file storage, and — for mobile — authorization enforcement via RLS

NO FOURTH TIER

There is no custom backend service, no queue, no cron worker, and no Supabase Edge Function in the deployed path. Business logic lives either in the client, in Next.js route handlers, or as Postgres constraints and RLS policies. This keeps the operational footprint to two deployables.

2.1 Mobile application — screen inventory

The mobile app carries two generations of UI simultaneously. A legacy shared stack serves brokers and any unmigrated flow; two dedicated, redesigned navigators serve the two primary revenue roles.

Navigator	Entry	Screens
Auth stack	Unauthenticated	Landing, Register, SignIn, OTP
ShipperNavigator REDESIGNED	role = shipper	Four-tab shell — ShipperHome, ShipperMyLoads, ShipperTrucks, ShipperProfile — plus a pushed ShipperPostLoad screen
TruckOwnerNavigator REDESIGNED	role = truck_owner	Four-tab shell — TruckOwnerHome, TruckOwnerAvailability, TruckOwnerLoads, TruckOwnerProfile — plus pushed PostAvailability, Trucks, AddTruck, EditTruck
Legacy app stack	Other roles / fallback	RoleDashboard, UploadLoad, ViewLoads, ViewAvailabilities, ManageTrucks, AddTruck, EditTruck, CreateAvailability, ManageAvailabilities, FindReturnLoad, Profile

Both redesigned navigators render their tab body by conditional mount rather than by a `Tab.Navigator`, and both share a single `ShipperTabBar` component — the truck-owner navigator imports the shipper's tab bar directly. Tab state is local component state, so tab selection does not survive a navigator remount.

2.2 Dashboard — route inventory

Route Group	Access	Pages
/login	Public	Email/password sign-in; role lookup then redirect. Shows dev bypass buttons when enabled.
/admin/*	role = admin	dashboard, loads, availabilities, alerts, users + users/[id], shippers, truck-owners, trucks + trucks/[id], locations, verifications, add-moderator, activity-log
/moderator/*	role = moderator	dashboard, loads, availabilities, users + users/[id], shippers, truck-owners, trucks + trucks/[id], verifications — no alerts, no leads, no settings
/api/admin/*	admin or moderator	9 route handlers — see section 09

03 Technology Stack — Full Inventory

3.1 Mobile application

Layer	Package	Version	Role in the system
Runtime	expo	~54.0.35	Build toolchain, native module management, OTA-capable runtime
	react-native	0.81.5	Native UI runtime; New Architecture explicitly disabled
	react	19.1.0	Component model
Navigation	@react-navigation/native	^7.1.34	Navigation container and state
	@react-navigation/native-stack	^7.14.6	Native stack navigators (auth, shipper, truck owner, legacy)
Native deps	react-native-screens	~4.16.0	Native screen primitives backing the stacks
	react-native-safe-area-context	~5.6.2	Notch/inset handling for the custom tab bar
	react-native-gesture-handler	~2.28.0	Gesture layer required by navigation
State	zustand	^5.0.12	Single global auth store — session, public user profile, resolved dashboard route
Backend client	@supabase/supabase-js	^2.99.3	PostgREST, Auth, and Storage client — the only network layer in the app
	react-native-url-polyfill	^3.0.0	URL/URLSearchParams polyfill required by supabase-js on Hermes
Persistence	@react-native-async-storage/async-storage	2.2.0	Session storage adapter for Supabase Auth; also backs local search history
Device / UI	@expo/vector-icons	^15.0.3	Ionicons + MaterialCommunityIcons used across tab bars and cards
	expo-font	~14.0.12	Custom font loading
	expo-document-picker	~14.0.8	Selecting KYC / vehicle documents for upload
	@react-native-community/datetimepicker	8.4.4	Loading dates and availability windows
Connectivity	@react-native-community/netinfo	11.4.1	Drives the offline banner and network-status hook
Tooling	typescript / @types/react	^5.9.3 / ~19.1.10	Types for the TS files (store, config, utils); most screens remain .js

NOTABLE ABSENCES

No data-fetching cache (React Query / SWR), no navigation-level state persistence, no error-reporting SDK (Sentry or equivalent), no analytics SDK, and no push-notification module. Data fetching is hand-rolled per screen against the Supabase client; observability is limited to a local `lib/logger.js` and an `ErrorBoundary` component.

3.2 Admin dashboard

Layer	Package	Version	Role in the system
Framework	next	16.2.2	App Router — server components, route handlers, edge middleware
	react / react-dom	19.2.4	UI runtime
	typescript	^5	Strict typing across the dashboard — it is fully TypeScript
Backend client	@supabase/supabase-js	^2.101.1	Service-role client on the server; anon client in the browser for login
	@supabase/ssr	^0.10.0	Cookie-based session bridging for server components and middleware
UI system	tailwindcss + @tailwindcss/postcss	^4	Utility styling; the design-token layer for the "LoadKaro Dispatch" theme
	shadcn	^4.1.2	Component generator — 11 primitives vendored into <code>components/ui</code>
	@base-ui/react	^1.3.0	Unstyled accessible primitives beneath the shadcn layer
	lucide-react	^1.7.0	Icon set
	class-variance-authority, clsx, tailwind-merge	—	Variant and class-composition helpers used by every UI primitive
	tw-animate-css	^1.4.0	Animation utilities (KPI counters, transitions)
Data grid	@tanstack/react-table	^8.21.3	Headless table engine powering <code>CommonDataTable</code> — every ops screen
Linting	eslint + eslint-config-next	^9 / 16.2.2	Static checks

3.3 Backend

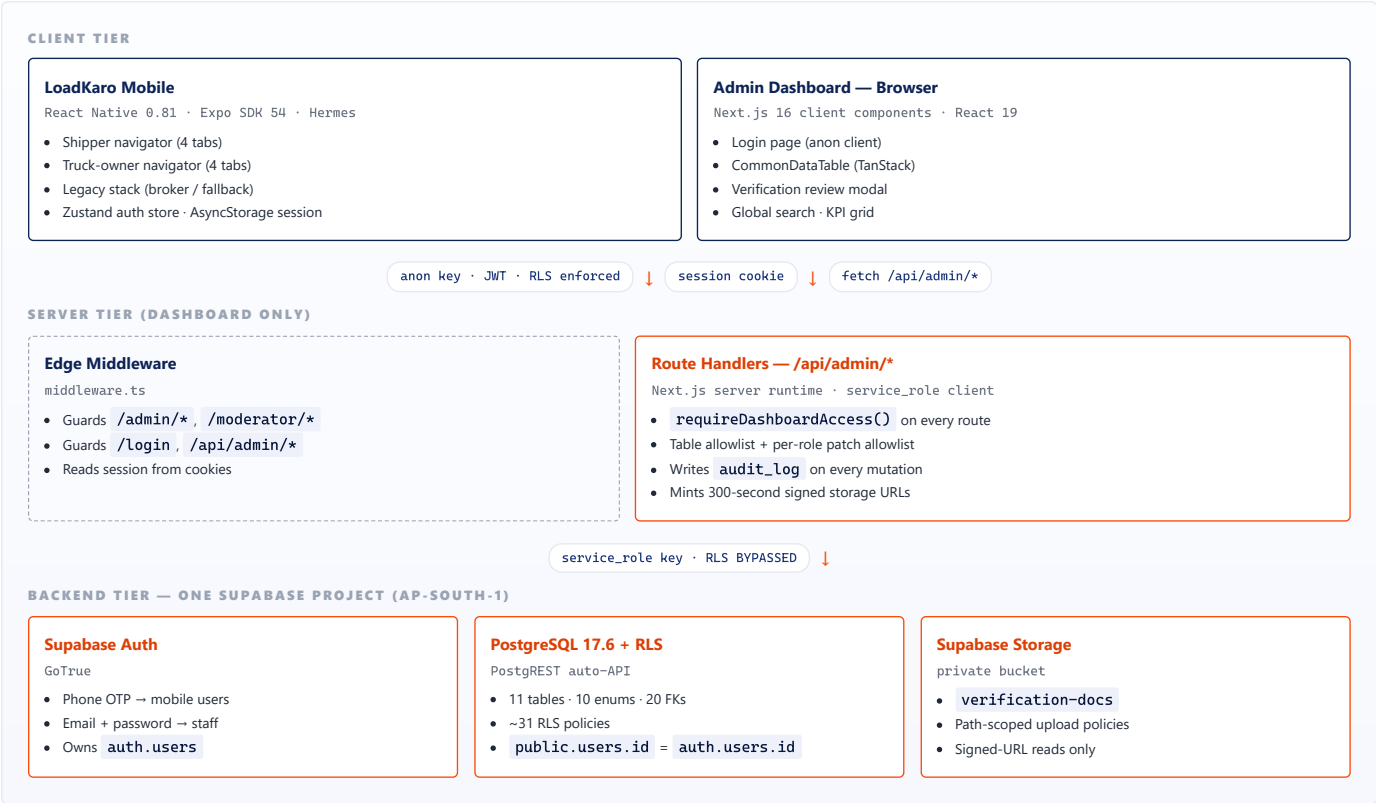
COMPONENT	TECHNOLOGY	CONFIGURATION
Database	PostgreSQL 17.6.1	11 application tables in <code>public</code> ; 10 custom enum types; ~41 indexes; RLS enabled on every table
API layer	PostgREST (Supabase auto-API)	Both clients speak PostgREST directly. Embedded joins are used heavily, e.g. <code>users!trucks_owner_id_fkey(name)</code>
Authentication	Supabase Auth (GoTrue)	Phone/SMS OTP for marketplace users; email + password for dashboard staff. Both write into the same <code>auth.users</code> table
Storage	Supabase Storage	One private bucket, <code>verification-docs</code> , keyed by <code>{entityType}/{entityId}/{submissionId}/{docKey}/{file}</code>
Authorization	Postgres RLS	~31 policies on <code>public</code> tables plus storage policies. This is the sole security boundary for the mobile app
Region	ap-south-1 (Mumbai)	Co-located with the Indian user base
Schema management	Manual	The project does not use Supabase CLI migrations. Schema and policy changes are applied directly, with ad-hoc SQL backups (<code>rls_backup_*.sql</code>) kept in the repo root

3.4 Cross-cutting

CONCERN	IMPLEMENTATION
Monorepo	npm workspaces at the repository root. One <code>npm install</code> ; per-app scripts via <code>npm --workspace <app> run <script></code>
Internationalization	Hand-rolled — a React context (<code>i18n/LanguageContext.js</code>) over four flat dictionaries (<code>en</code> , <code>hi</code> , <code>kn</code> , <code>te</code>), roughly 355 keys each. No <code>i18n</code> library, no pluralization engine, no lazy loading
Design tokens	Mobile: <code>theme/theme.js</code> — navy <code>#0C2356</code> + orange <code>#FB4B0A</code> sampled from the logo. Dashboard: the parallel "LoadKaro Dispatch" token set with an azure brand and dark mode
Network resilience	Mobile wraps every Supabase request in a 30-second <code>AbortController</code> timeout, because React Native's <code>fetch</code> has no default read timeout and a stalled 2G socket would otherwise hang a screen indefinitely
Audit	Every dashboard mutation writes an <code>audit_log</code> row, fire-and-forget

04 System Topology

The following is the complete set of runtime connections in the platform. Every arrow is a real network path; there are no others.



05 Connection Contracts

"How it's connected" is best understood as four distinct contracts. Each has its own credential, its own enforcement point, and its own failure mode.

5.1 Contract A — Mobile ↔ Supabase (direct, RLS-governed)

PROPERTY	DETAIL
Client construction	A single module-level client in <code>LoadKaro/Lib/supabase.js</code> . There is exactly one client instance in the app; every screen imports it.
Credentials	<code>EXPO_PUBLIC_SUPABASE_URL</code> + <code>EXPO_PUBLIC_SUPABASE_ANON_KEY</code> (falling back to the legacy <code>EXPO_PUBLIC_SUPABASE_KEY</code>). Both are compiled into the bundle and are public by design.
Session handling	<code>storage: AsyncStorage, autoRefreshToken: true, persistSession: true, detectSessionInUrl: false</code> . Sessions survive app restarts.
Transport hardening	A custom <code>fetch</code> wrapper aborts any request after 30 seconds unless the caller supplied its own signal. This exists specifically for users on stalled 2G connections, where the socket stays open but no data flows.
Degraded mode	If env vars are missing, <code>isSupabaseConfigured</code> is <code>false</code> and a placeholder client is constructed against <code>https://placeholder.supabase.co</code> so the app fails at the request level rather than crashing at import.
Query style	Direct PostgREST calls with embedded joins and <code>.range()</code> pagination at <code>PAGE_SIZE = 20</code> . No client-side cache — every screen re-queries on focus.
Failure mode	An RLS mismatch is <i>silent</i> on reads: forbidden rows are simply absent from the result set, so a policy regression reads as "no data" rather than an error.

5.2 Contract B — Dashboard browser ↔ Next.js server

PROPERTY	DETAIL
Transport	Ordinary <code>fetch</code> from client components to same-origin <code>/api/admin/*</code> route handlers.
Credential	The Supabase session cookie, written at login by <code>@supabase/ssr</code> and read server-side by <code>createSupabaseServerAuthClient()</code> .
First gate	<code>middleware.ts</code> matches <code>/admin/:path*</code> , <code>/moderator/:path*</code> , <code>/login</code> , and <code>/api/admin/:path*</code> ; unauthenticated or wrong-role page requests are redirected to <code>/login?next=...</code>
Second gate	Every route handler independently calls <code>requireDashboardAccess()</code> , which re-reads the session, re-queries <code>public.users.role</code> , and accepts only <code>admin</code> or <code>moderator</code> . Middleware alone is never trusted.
Third gate	<code>hasServiceRoleKey()</code> — if the key is absent the route returns <code>503</code> with setup instructions rather than failing obscurely.

5.3 Contract C — Next.js server ↔ Supabase (service role)

PROPERTY	DETAIL
Client construction	<code>createServiceRoleClient()</code> in <code>src/lib/supabase/service-role.ts</code> , with <code>persistSession: false</code> and <code>autoRefreshToken: false</code> .
Credential	<code>SUPABASE_SERVICE_ROLE_KEY</code> — server-scoped only, never prefixed with <code>NEXT_PUBLIC_</code> , and therefore never shipped to the browser.
Privilege	Full read/write on every table. RLS does not apply.
Compensating controls	(1) table allowlists per HTTP verb; (2) <code>admin_alerts</code> restricted to <code>admin</code> and never deletable; (3) <code>filterPatchByAllowlist()</code> reduces every PATCH body to the columns that role may touch on that table; (4) <code>phone</code> and <code>email</code> stripped from every patch unconditionally; (5) <code>select</code> parameter validated against <code> /^[^\w\-,.*!()]+\$/ </code> and capped at 8 000 characters; (6) an <code>audit_log</code> row per mutation.

5.4 Contract D — The shared schema (the real integration point)

The two applications never call each other. Their only coupling is the database schema — which makes the schema an **undeclared, unversioned API contract** between them.

SHARED OBJECT	MOBILE DEPENDS ON IT FOR	DASHBOARD DEPENDS ON IT FOR
<code>users.role</code>	Routing the user to a navigator after OTP	Authorizing dashboard access at both gates
<code>users.verification_status</code>	Rendering the verification badge and gating flows	The write target of every verification decision
<code>trucks.verification_status</code>	Blocking edits once a truck is <code>pending</code> or <code>verified</code>	The write target of truck verification decisions
<code>loads / availabilities.status</code>	Filtering market listings	Ops close/cancel actions
<code>interests</code>	Written when a user taps "Show Interest"	Read as the lead pipeline; <code>status</code> advanced by staff
<code>verification_submissions / _documents</code>	Written on upload; read in "My Documents"	Read for review; updated with the decision
All 10 enum types	Hardcoded in <code>config/*.ts</code> label maps	Mirrored in <code>src/lib/schema/enums.ts</code>

CONTRACT RISK

Enum values are duplicated in three places — Postgres, the mobile label configs, and the dashboard's `enums.ts`. Because there are no migration files and no generated types wired into either app, adding an enum value in the database will not produce a compile error anywhere; it will surface as an unlabelled value in the UI. This is the single highest-friction point in the platform's connective tissue.

06 Identity & Authentication

Both applications authenticate against the same `auth.users` table, but through different methods and with different post-login resolution.

6.1 Mobile — phone OTP

Two modes share one OTP screen. **Register** collects name, phone, and role up front and passes them as auth metadata; **Sign in** collects only the phone number.

- 1 The screen normalizes the number via `formatPhoneOtpIndia()` — strips non-digits, accepts 10-digit, `91`-prefixed 12-digit, or `0`-prefixed 11-digit input, validates the national part against `/^[6-9]\d{9}$/`, and emits E.164 `+91XXXXXXXXXX`.
- 2 `supabase.auth.signInWithOtp({ phone, ... })` triggers the SMS.
- 3 The user enters six digits; `verifyOtp({ phone, token, type: 'sms' })` returns a session.
- 4 `syncUserAfterOtp()` reconciles `auth.users` with `public.users`: on register, if no profile row exists it inserts one (`id`, `name`, `phone`, `role`); it then selects the full profile.
- 5 `resolveDashboardRoute()` maps `role` to a navigator via `config/roleRoutes.ts`.
- 6 The Zustand store receives `{ user, session, dashboardRoute }`, and `RootNavigator` — keyed on `dashboardRoute` — remounts into the correct experience.

SESSION LIFECYCLE

- `initializeAuth()` runs on mount: `getSession()`, fetch the public profile if a session exists, set `isReady`, then install an `onAuthStateChange` listener.
- The listener debounces its profile fetch by **400 ms** to avoid racing the profile `INSERT` that registration performs moments earlier.
- If a profile fetch fails but a user is already in state, the existing user is preserved — transient network errors do not log the user out.
- `signOut()` clears Supabase state, clears the store, and resets the route to the default.

OPERATIONAL DEPENDENCY

Mobile sign-up and sign-in are entirely dependent on SMS delivery. In the Indian market this means DLT registration of the sender entity, header, and templates. No fallback authentication method (email, magic link, WhatsApp) exists in the app, so an SMS outage is a total outage of new-user acquisition and of any user without a live session.

6.2 Dashboard — email and password

- 1 `signInWithPassword({ email, password })` against the browser anon client.
- 2 Immediately after, the page reads `role` from `public.users` for that user id.
- 3 `admin` → `/admin/dashboard`; `moderator` → `/moderator/dashboard`; anything else → forced `signOut()` plus a "not authorized" message. A marketplace user who somehow knows a dashboard URL cannot hold a session there.
- 4 Every subsequent request is re-checked by middleware and again by `requireDashboardAccess()` inside each route handler.

MODERATOR PROVISIONING

Moderators are created by an admin through a two-step OTP flow rather than by password invitation: `POST /api/admin/moderator/send-otp` (validates the phone, rejects duplicates with **409**, sends an OTP) followed by `POST /api/admin/moderator/create` (verifies the OTP, then either upgrades the existing `users` row to `moderator` or inserts a new row already marked `verified`). New inserts write a `create_moderator` audit entry.

6.3 Development bypass

For local work without real Supabase Auth users, `DASHBOARD_BYPASS_AUTH` (server) and `NEXT_PUBLIC_DASHBOARD_BYPASS_AUTH` (client) short-circuit both gates. The role is read from an `lk-bypass-role` cookie, defaulting to `admin`, and `requireDashboardAccess()` returns `{ ok: true, bypass: true, userId: null }` — which means audit rows written in bypass mode have a null actor.

PRODUCTION HAZARD

With bypass enabled, anyone who can reach the dashboard origin has full `service_role`-backed access to every record. These flags must be absent — not merely false — in every production environment, and should be part of the deployment checklist.

07 Data Model

Eleven application tables in the `public` schema, plus Supabase's own `auth` and `storage` schemas. RLS is enabled on all eleven.

7.1 Table catalogue

TABLE	GROUP	PURPOSE AND KEY COLUMNS
<code>users</code>	Identity	Application profile, 1:1 with <code>auth.users</code> (PK is also the FK, ON DELETE CASCADE). Holds <code>role</code> , <code>verification_status</code> , <code>subscription_type</code> , <code>phone</code> is unique.
<code>trucks</code>	Supply	A vehicle owned by a truck owner. Physical specs (<code>capacity_tons</code> , axle/wheel counts, internal dimensions), <code>permit_type</code> , <code>gps_available</code> , and its own <code>verification_status</code> .
<code>truck_variants</code>	Reference	Lookup of 31 seeded variant names grouped by <code>truck_category_enum</code> .
<code>availabilities</code>	Supply	A truck owner advertising capacity: truck, origin/destination, <code>available_from/_till</code> (CHECK-constrained ordering), <code>expected_rate</code> , live lat/long, and <code>status</code> .
<code>loads</code>	Demand	A shipper's freight: origin/destination (ON DELETE RESTRICT), <code>loading_date</code> , <code>truck_category_required</code> , <code>capacity_required</code> , <code>payment_type</code> , <code>advance_percentage</code> , optional rate, <code>status</code> .
<code>locations</code>	Reference	2 166 pre-seeded Indian cities with state and optional coordinates. Every route field on both sides of the marketplace points here.
<code>interests</code>	Marketplace	The lead record. Points at either a <code>load_id</code> or an <code>availability_id</code> , plus <code>interested_by</code> and <code>contacted_party</code> . <code>status</code> advances <code>interested</code> → <code>contacted</code> → <code>matched</code> , or <code>expired</code> .
<code>verification_submissions</code>	Trust	One review request for an entity (<code>entity_type</code> is <code>user</code> or <code>truck</code>). Carries the decision, reviewer, timestamp, and rejection reason.
<code>verification_documents</code>	Trust	One row per uploaded file: <code>doc_key</code> , <code>bucket</code> , <code>path</code> , original filename, MIME type, size. Cascades from its submission.
<code>admin_alerts</code>	Ops	One alert per posted load (<code>load_id</code> is UNIQUE), with <code>is_handled</code> , <code>handled_by</code> , <code>handled_at</code> .
<code>audit_log</code>	Compliance	Append-only record of every dashboard mutation: actor, role, action, entity, and a <code>jsonb</code> details payload.

7.2 Relationship map

```

auth.users
├── 1:1→ public.users (id = id, CASCADE)
│   ├── 1:N→ trucks (owner_id, CASCADE)
│   │   └── 1:N→ availabilities (truck_id, CASCADE)
│   ├── 1:N→ availabilities (owner_id, CASCADE)
│   ├── 1:N→ loads (posted_by, CASCADE)
│   │   └── 1:1→ admin_alerts (load_id UNIQUE, CASCADE)
│   └── 0:N→ admin_alerts (handled_by)
locations
├── loads.origin_location_id / destination_location_id (RESTRICT)
└── availabilities.origin_location_id / destination... (NO ACTION)
truck_variants → trucks.variant_id (NO ACTION)
interests → loads.id | availabilities.id + auth.users *2 (NO ACTION)
verification_submissions 1:N→ verification_documents (CASCADE)
└── auth.users (submitted_by, reviewed_by)
audit_log → auth.users (actor_id)

```

Two deletion behaviours are worth noting. Deleting a user cascades away their trucks, loads, and availabilities — but **not** their `interests` rows, which reference `auth.users` with `NO ACTION`. And `loads` reference `locations` with `RESTRICT`, so a seeded city that has ever been used cannot be deleted.

7.3 Enum types

ENUM	VALUES
<code>user_role_enum</code>	<code>truck_owner</code> , <code>broker</code> , <code>shipper</code> , <code>moderator</code> , <code>admin</code>
<code>user_verification_status_enum</code>	<code>unverified</code> , <code>verified</code> , <code>rejected</code> , <code>pending</code>
<code>truck_verification_status_enum</code>	<code>unverified</code> , <code>verified</code> , <code>rejected</code> , <code>pending</code>
<code>truck_category_enum</code>	<code>open</code> , <code>container</code> , <code>lcv</code> , <code>mini_pickup</code> , <code>trailer</code> , <code>tipper</code> , <code>tanker</code> , <code>dumper</code> , <code>bulker</code>
<code>permit_type_enum</code>	<code>national_permit</code> , <code>state_permit</code> , <code>all_india_permit</code> , <code>goods_carriage</code> , <code>contract_carriage</code>
<code>truck_availability_status_enum</code>	<code>available</code> , <code>closed</code> , <code>cancelled</code>
<code>load_status_enum</code>	<code>open</code> , <code>matched</code> , <code>cancelled</code> , <code>closed</code>
<code>payment_type_enum</code>	<code>advance</code> , <code>partial_advance</code> , <code>after_delivery</code>
<code>subscription_type_enum</code>	<code>free</code> , <code>premium</code> , <code>enterprise</code> DEFINED, NOT MONETIZED

ENUM	VALUES
load_rate_visibility_enum	visible, hidden ORPHANED — NO COLUMN USES IT

interests.status and the verification tables' status / entity_type / review_decision are text columns constrained by CHECK rather than true enums.

7.4 Business rules enforced in the database

CONSTRAINT	TABLE	RULE
check_advance_logic	loads	advance requires advance_percentage = 100; partial_advance requires 1–99; after_delivery requires NULL. Payment semantics cannot be corrupted by a buggy client.
loads_advance_percentage_check	loads	If present, must be 1–100.
check_dates	availabilities	available_till must be NULL or ≥ available_from.
users_phone_key	users	One account per phone number.
admin_alerts_load_id_key	admin_alerts	At most one alert per load.

Availability overlap prevention — stopping a truck being double-booked across two date windows — is enforced only in the client, by a query performed before insert. It is not a database constraint, so a concurrent double submission can defeat it.

7.5 Index strategy

Roughly 41 indexes support the two dominant access patterns. Marketplace browsing is served by route-and-date composites (idx_loads_route_date over origin, destination, loading date), a partial index on open loads only (idx_open_loads WHERE status = 'open'), and status/date indexes on availabilities. Ops screens are served by created_at DESC composites on audit_log , interests , and the verification tables — matching the dashboard's default newest-first sort.

08 Core Data Flows

8.1 Posting a load (shipper)

- 1 The post screen loads the `locations` reference data for state and city pickers.
- 2 The shipper fills route, loading date, required truck category, capacity, and payment terms.
- 3 The client derives `advance_percentage` from `payment_type` — 100 for `advance`, the user's figure for `partial_advance`, NULL for `after_delivery` — mirroring the database CHECK so the insert cannot be rejected for that reason.
- 4 `INSERT INTO loads` with `posted_by = auth.uid()`; `status` defaults to `open`.
- 5 An `admin_alerts` row is created for the load, surfacing it on the admin alerts screen.
- 6 The shipper is returned to their loads list.

8.2 Browsing and expressing interest (either side)

The market list screens issue a single PostgREST query with embedded joins — pulling the poster's name and phone and both location rows in one round trip — filtered by route and date, ordered newest first, and paginated with `.range()` at 20 rows per page. Two actions are available on each card:

ACTION	EFFECT	NOTES
Call	<code>Linking.openURL('tel:' + phone)</code>	Hands off to the device dialer. The platform records nothing about the call — no duration, no outcome, no confirmation it happened.
Show Interest	<code>INSERT INTO interests</code>	Writes <code>load_id</code> or <code>availability_id</code> , <code>interested_by</code> , <code>contacted_party</code> , and <code>status = 'interested'</code> . This is the platform's only liquidity signal.

8.3 Posting availability (truck owner)

- 1 Load the owner's own trucks plus the location reference data.
- 2 Owner selects a truck, an origin, up to two preferred destinations, and a date window.
- 3 Client-side validation rejects an inverted range or a start date in the past.
- 4 An overlap query checks for an existing availability on the same truck covering those dates.
- 5 `INSERT INTO availabilities` with `status = 'available'`.

8.4 Verification — the one flow that crosses both applications

This is the only workflow where a mobile action is completed by a dashboard action. It is therefore the most important interconnection in the platform.



The decision handler performs two sequential updates — the entity's `verification_status`, then the submission's `status`, `review_decision`, `reviewed_at`, `reviewed_by`, and any `rejection_reason` — followed by a fire-and-forget audit write. The two updates are not wrapped in a transaction, so a failure between them would leave the entity and its submission disagreeing.

Back on mobile, the user's "My Documents" view reads their most recent submission and its document rows, and opens each file through a 300-second signed URL generated by the anon client. A rejected user can resubmit, which creates a new submission and returns the entity to `pending`.

8.5 Dashboard record operations

Every ops screen is the same `CommonDataTable` component pointed at a different table. It reads through the single generic `GET /api/admin/data` endpoint and writes through `PATCH /api/admin/row`, which means adding a new ops screen requires no new API surface — only a column configuration. Inline verification changes are permitted directly from the grid when the current status is `pending` or `unverified`.

09 Dashboard API Surface

Nine route handlers, all under `/api/admin/`, all requiring `requireDashboardAccess()` and a present service-role key.

VERB	ROUTE	ACCESS	BEHAVIOUR
GET	<code>/data</code>	admin, mod	Generic paginated read for any allowlisted table. Accepts <code>table</code> , <code>select</code> , <code>page</code> , <code>pageSize</code> (max 100), <code>sortColumn</code> , <code>sortAsc</code> , <code>searchColumn/searchValue</code> (UUID → eq, else <i>like</i>), a JSON <code>filters</code> map, and a <code>dateColumn/dateFrom/dateTo</code> range. Returns <code>{ data, count }</code> . <code>admin_alerts</code> is admin-only.
PATCH	<code>/row</code>	admin, mod	Updates one row after reducing the patch to the role's allowlist for that table. Special-cases <code>admin_alerts.is_handled</code> by stamping <code>handled_at</code> and <code>handled_by</code> . Derives a specific audit action from the payload. Returns 400 if no field survives the allowlist.
DELETE	<code>/row</code>	ADMIN	Deletes one row from <code>loads</code> , <code>availabilities</code> , <code>trucks</code> , <code>users</code> , or <code>interests</code> . <code>admin_alerts</code> can never be deleted. Writes <code>delete_record</code> .
GET	<code>/search</code>	admin, mod	Global search (min 2 characters) across users, trucks, and loads in parallel, five results each, returning a normalized <code>{ type, id, title, subtitle }</code> list.
POST	<code>/moderator/send-otp</code>	ADMIN	Validates <code>/^\\+\\d{10,15}\$</code> , rejects an existing phone with 409, sends the OTP.
POST	<code>/moderator/create</code>	ADMIN	Verifies the OTP, then upgrades or inserts the <code>users</code> row as <code>moderator</code> .
GET	<code>/verification/submissions</code>	admin, mod	Submissions for an <code>entity_type</code> , optionally narrowed by <code>entity_id</code> or <code>status</code> , newest first.
GET	<code>/verification/documents</code>	admin, mod	Document rows for one <code>submission_id</code> , oldest first.
POST	<code>/verification/signed-url</code>	admin, mod	Mints a 300-second signed URL. Only the <code>verification-docs</code> bucket is accepted.
POST	<code>/verification/decision</code>	admin, mod	Applies <code>verified</code> or <code>rejected</code> to the entity and closes the submission.

9.1 Standard status codes

CODE	MEANING IN THIS API
400	Malformed parameters, or a patch whose every field was stripped by the allowlist
401	No valid session
403	Session exists but the role is not <code>admin/moderator</code> , or the role may not touch that table
409	Moderator creation against a phone number already in <code>users</code>
503	<code>SUPABASE_SERVICE_ROLE_KEY</code> is not configured — returned with setup guidance

9.2 Audit actions

ACTION	TRIGGER
<code>update_record</code>	Any PATCH with no more specific interpretation
<code>delete_record</code>	Any DELETE
<code>verify_user / reject_user</code>	PATCH of <code>users.verification_status</code>
<code>verify_truck / reject_truck</code>	PATCH of <code>trucks.verification_status</code>
<code>mark_alert_handled</code>	PATCH <code>admin_alerts.is_handled = true</code>
<code>close_record</code>	PATCH of any <code>status</code> to <code>closed</code>
<code>create_moderator</code>	New moderator inserted (not on role upgrade)
<code>review_verification</code>	Verification decision endpoint

AUDIT DURABILITY

Audit writes are fire-and-forget (`void writeAuditLog(...)`). A failed write is logged to the console but never blocks or fails the request, so the audit trail is best-effort rather than guaranteed. Under bypass mode the actor id is null.

10 Security & Authorization

10.1 Two enforcement systems

	MOBILE PATH	DASHBOARD PATH
Credential	anon key + per-user JWT	service_role key (server-side only)
Enforced by	Postgres RLS policies	TypeScript in middleware and route handlers
Enforced where	Inside the database, per row	Before the query is issued
Failure mode	Silent — rows disappear from results	Loud — 401/403, or catastrophic if a check is omitted
Reviewability	Requires SQL inspection of live policies	Visible in code review and diffable in git

10.2 Access model by role

CAPABILITY	SHIPPER	TRUCK OWNER	BROKER	MODERATOR	ADMIN
Post loads	✓	—	—	—	—
Post availabilities	—	✓	—	—	—
Manage trucks	—	✓	—	—	—
Browse the other side	✓ availabilities	✓ loads	✓ both (read-only)	via dashboard	via dashboard
Call / show interest	✓	✓	—	—	—
Upload own KYC	✓	✓	✓	—	—
Review verifications	—	—	—	✓	✓
Edit records	own only	own only	—	status fields only	full allowlist
Delete records	own only	own only	—	✗	✓ (not alerts)
Leads, alerts, activity log	—	—	—	✗	✓
Create moderators	—	—	—	✗	✓

10.3 Representative RLS policies

TABLE	COMMAND	CONDITION
users	SELECT	Two policies coexist: <code>auth.uid() = id</code> for the own row, and a permissive <code>true</code> making the user directory readable — which is what enables counterparty names and phone numbers to appear on market cards.
	INSERT	<code>auth.uid() = id</code> — a user may only create their own profile row
	UPDATE	<code>auth.uid() = id</code>
trucks	SELECT	Ownership OR staff role OR <code>true</code> — the trailing <code>true</code> makes the whole fleet publicly readable to any authenticated user
	INSERT	<code>owner_id = auth.uid()</code> AND the caller's role is <code>truck_owner</code>
	UPDATE	Owner may edit <i>only while</i> <code>verification_status = 'unverified'</code> ; admins may always edit — this is what locks a truck's specs once documents are under review
	DELETE	Owner or admin
truck_variants, locations	SELECT	Open reference data

PERMISSIVE-CLAUSE PATTERN

Several read policies terminate in `OR true`, which makes the preceding ownership and role checks decorative for SELECT. This is deliberate for a marketplace where both sides must see each other's listings and contact details — but it means *any* authenticated account can enumerate the full user and truck directory, including phone numbers. That is a scraping and competitor- intelligence exposure, not a bug in the code.

10.4 Dashboard patch allowlist

TABLE	ADMIN MAY PATCH	MODERATOR MAY PATCH
users	name, role, verification_status, subscription_type	verification_status
loads	loading_date, truck_category_required, capacity_required, payment_type, advance_percentage, rate_optional, status, both location ids	status

TABLE	ADMIN MAY PATCH	MODERATOR MAY PATCH
trucks	category, variant_id, capacity_tons, gps_available, permit_type, verification_status	verification_status
availabilities	available_from, available_till, expected_rate, status, current_location, both preferred destinations, both location ids	status
admin_alerts	is_handled	—
interests	status	status

phone and email are stripped from every patch body before any of this runs, on every table, for every role — staff cannot rewrite a user's contact identity through the dashboard.

10.5 Injection surface

The one place user-controlled text reaches PostgREST's query grammar is the select parameter of GET /api/admin/data , which must support embedded joins. It is defended by a character allowlist (/^[\\w\\-,.*!() :]+\$/) and an 8 000-character cap; sort columns are restricted to /^[\\w]+\$/ . Everywhere else, values pass through the Supabase client's parameterized builders.

11 Storage Subsystem

PROPERTY	DETAIL
Bucket	<code>verification-docs</code> — private; no public URLs are ever issued
Path convention	<code>{entityType}/{entityId}/{submissionId}/{docKey}/{timestamp}-{filename}</code>
Write path	Mobile only, via the authenticated anon client. Storage policies scope writes to the caller's own entity prefix.
Upload mechanics	The picked file URI is fetched into a blob, converted to an <code>ArrayBuffer</code> , and uploaded with an explicit content type — the reliable path on React Native, where streaming uploads are not available.
Read path (mobile)	Anon client <code>createSignedUrl(path, 300)</code> , then handed to the OS via <code>Linking.openURL</code> .
Read path (dashboard)	<code>POST /api/admin/verification/signed-url</code> using the service-role client, with the bucket name checked against a single-entry allowlist.
URL lifetime	300 seconds on both paths
Metadata	Every file has a <code>verification_documents</code> row carrying bucket, path, original filename, MIME type, and byte size — so the database, not the bucket listing, is the index.
Retention	No deletion or retention policy is implemented. Documents from superseded submissions remain in the bucket indefinitely.

DATA-PROTECTION NOTE

This bucket holds Aadhaar, PAN, driving licences, GST certificates, and vehicle registration documents — the most sensitive data the platform touches. The controls in place (private bucket, path-scoped writes, short-lived signed reads, full audit of every review) are appropriate. The open items are retention and deletion, which are policy decisions rather than engineering gaps.

12 Configuration & Environments

12.1 Mobile — LoadKaro/.env

VARIABLE	REQUIRED	PURPOSE
EXPO_PUBLIC_SUPABASE_URL	Yes	Project endpoint
EXPO_PUBLIC_SUPABASE_ANON_KEY	Yes	Public anon key
EXPO_PUBLIC_SUPABASE_KEY	No	Legacy alias, used as a fallback
EXPO_PUBLIC_TEST_MODE	No	Set to "false" in every EAS build profile

Only `EXPO_PUBLIC_`-prefixed variables reach the bundle. Everything in this file is public by construction — no secret may be placed here.

12.2 Dashboard — admin-dashboard/.env.local

VARIABLE	SCOPE	PURPOSE
NEXT_PUBLIC_SUPABASE_URL	Client + server	Project endpoint
NEXT_PUBLIC_SUPABASE_ANON_KEY	Client + server	Browser login client
NEXT_PUBLIC_SUPABASE_KEY	Client + server	Alias fallback
SUPABASE_SERVICE_ROLE_KEY	SERVER ONLY	RLS-bypassing key for all <code>/api/admin/*</code> operations. Absent → every route returns 503.
DASHBOARD_BYPASS_AUTH	Server	DEV ONLY — skips auth
NEXT_PUBLIC_DASHBOARD_BYPASS_AUTH	Client + server	DEV ONLY — shows bypass buttons

12.3 Shared-env bridge

`next.config.ts` maps `EXPO_PUBLIC_SUPABASE_URL`, `_ANON_KEY`, and `_KEY` onto their `NEXT_PUBLIC_` equivalents, and the middleware falls back to the `EXPO_PUBLIC_` names when the Next ones are unset. A single `.env` can therefore serve both applications during development. This convenience must not be extended to the service-role key, which has no `EXPO_PUBLIC_` counterpart and must never acquire one.

12.4 Environment topology

SINGLE-ENVIRONMENT REALITY

There is one Supabase project. No staging or preview project is configured, and no Supabase branch workflow is in use. Combined with the absence of migration files, this means schema and policy changes are made directly against the production database, with safety resting on manual `rls_backup*.sql` snapshots kept in the repository root. This is the platform's largest operational risk and the clearest candidate for hardening.

13 Build, Release & Deployment

13.1 Mobile — EAS Build

PROFILE	ARTIFACT	DISTRIBUTION	USE
development	Dev client	Internal	Local development against a custom native runtime
preview	APK	Internal	Sideloaded QA builds
production	AAB	Store	Google Play submission
production-apk	APK	Direct	Direct distribution outside the store

BUILD-CRITICAL CONFIGURATION

SETTING	VALUE	WHY IT MATTERS
newArchEnabled	false	The React Native New Architecture is explicitly disabled; enabling it previously caused APK launch crashes.
Android package	in.loadkaro.app	Store identity — immutable once published
appVersionSource	remote	EAS owns version-code increments
Expo plugins	datetimepicker, expo-font	The only native config plugins; a font duplication issue here previously caused launch failures
Icons	Adaptive foreground + monochrome	Themed-icon support on Android 13+
EAS project	10b724fb-..., owner chanakyagts-organization	Build account binding

13.2 Dashboard

Standard Nextjs: `npm --workspace admin-dashboard run build` then `... run start`. Because the service-role key must stay server-side and middleware runs on every matched request, the dashboard requires a Node server — it cannot be exported as a static site. No hosting configuration (Vercel, Docker, CI workflow) is committed to the repository, so deployment is currently a manual operation.

13.3 Backend

Managed by Supabase; nothing to deploy. Schema and policy changes are applied manually, as noted in section 12.4.

13.4 Release checklist implied by the architecture

- 1 Confirm `DASHBOARD_BYPASS_AUTH` and `NEXT_PUBLIC_DASHBOARD_BYPASS_AUTH` are unset in the deployed environment.
- 2 Confirm `SUPABASE_SERVICE_ROLE_KEY` is present server-side and absent from any client bundle.
- 3 Snapshot RLS policies to a dated `rls_backup_*.sql` before any policy change.
- 4 Verify SMS/DLT sender configuration — without it, no one can sign in.
- 5 Build the mobile app with `newArchEnabled: false` intact.
- 6 Smoke-test one full cross-application loop: upload a document on mobile, approve it on the dashboard, and confirm the badge changes on mobile.

14 Repository Layout

mobile App/ - package.json - README.md - rls_backup_2026-05-25.sql - rls_backup_2026-07-01.sql - reports/	npm workspaces root workspace definitions manual RLS snapshots – no migration system audits, reviews, and this document
LoadKaro/ - App.js · index.js - app.json · eas.json - navigation/ - screens/ - shipper/ - truckowner/ - components/ - config/ - store/authStore.ts - services/ - lib/ - hooks/ · utils/ · types/ - i18n/ - theme/theme.js - android/ · assets/ - DOCS_*.md	— EXPO MOBILE APP — Expo + build configuration RootNavigator, Shipper-, TruckOwner- legacy stack (17) + redesigned shipper screens (5) ... + redesigned truck-owner screens (8) shared UI + components/shipper/ tab bar role routes, label maps, field configs (TS) the only global state syncUserAfterOtp, loadUserProfile supabase client, logger, verificationUpload en · hi · kn · te + LanguageContext navy/orange brand tokens four in-repo reference documents
admin-dashboard/ - middleware.ts - next.config.ts - components.json - admin_dashboard_api_collection.json	— NEXT.JS DASHBOARD — first auth gate EXPO_PUBLIC_ → NEXT_PUBLIC_ bridge shadcn configuration
src/ - app/ - login/ - admin/ - moderator/ - api/admin/ - components/ - admin/ - dashboard/ - brand/apex.tsx - ui/ - lib/ - supabase/ - auth/ - admin/patch-allowlist.ts - schema/enums.ts - audit.ts	13 route groups 9 route groups 9 route handlers common-data-table, verification-docs-modal shell, stats grid, global search, theme toggle the logo motif 11 shadcn primitives client · server · server-auth · service-role · env dashboard-access · dashboard-role · bypass mirror of the Postgres enums

14.1 Where to look when changing something

IF YOU ARE CHANGING...	START HERE
Who can see or write what, on mobile	The RLS policies in Postgres — not the app code
Who can see or write what, on the dashboard	lib/auth/dashboard-access.ts and lib/admin/patch-allowlist.ts
Adding a database column or enum value	Postgres, then lib/schema/enums.ts, then the mobile config/*Labels.ts maps — all three, by hand
Adding a dashboard ops screen	A new page using CommonDataTable ; no new API route is usually required
Adding a mobile screen for shipper or truck owner	The role's navigator plus its screens/<role>/ directory
Any user-visible string	All four dictionaries in i18n/
Brand colours	theme/theme.js (mobile) and the Tailwind token layer (dashboard) — these are two separate palettes

15 Architectural Observations

Recorded as findings of the discovery, not as a remediation plan. Severity reflects the risk to the platform, not the effort to address it.

15.1 Structural strengths

- **One backend, one schema.** With no service mesh and no inter-app API, there is a single source of truth and a very small operational surface for a team of this size.
- **Defence in depth on the dashboard.** Middleware, a per-handler access check, a table allowlist, a per-role column allowlist, unconditional PII stripping, and an audit row — five independent controls in front of a key that bypasses the database's own security.
- **Business rules in the database.** The payment/advance logic is a CHECK constraint, so no client bug can produce a load with incoherent payment terms.
- **One generic data endpoint.** Every ops screen reuses `/api/admin/data` and `CommonDataTable`, so the audited attack surface stays constant as screens are added.
- **Field-realistic engineering.** The 30-second fetch timeout, the offline banner, the 400 ms auth-listener debounce, and four Indian languages are all responses to how the target user actually connects.

15.2 Findings

#	FINDING	SEVERITY	DETAIL
1	Production is the only environment	HIGH	One Supabase project, no migrations, no staging, no branch workflow. Schema and RLS changes are applied live, with only manual SQL snapshots as a rollback path.
2	Enum definitions triplicated	HIGH	Postgres, <code>lib/schema/enums.ts</code> , and the mobile label configs must be edited together. Nothing enforces it; drift surfaces as unlabelled values in production UI. Generated types exist as a Supabase capability but are not wired into either build.
3	Bypass flags are a live production hazard	HIGH	A single stray environment variable converts the dashboard into an unauthenticated, <code>service_role</code> -backed interface to the entire database.
4	Directory-wide readability	MEDIUM	<code>OR true</code> in the user and truck SELECT policies lets any authenticated account enumerate all users — names and phone numbers included — and all vehicles. Necessary for the marketplace to function as designed, but it is an open scraping surface.
5	Verification decision is not transactional	MEDIUM	Entity status and submission status are two sequential updates. A failure between them leaves them inconsistent, with no reconciliation process.
6	Audit trail is best-effort	MEDIUM	Fire-and-forget writes mean a mutation can succeed with no audit record. For a log intended to support compliance, this weakens the guarantee it exists to provide.
7	No error or usage telemetry	MEDIUM	No crash reporter and no analytics on either surface. A mobile crash in the field is invisible; funnel questions can only be answered by querying the database directly.
8	Availability overlap enforced client-side only	MEDIUM	A pre-insert query, not a constraint. Concurrent submissions can double-book a truck.
9	Single-channel authentication	MEDIUM	SMS OTP is the only way into the mobile app. An SMS or DLT disruption is a total acquisition outage with no fallback path.
10	Two UI generations coexist	LOW	Redesigned navigators for shipper and truck owner sit alongside the legacy stack that still serves brokers. The truck-owner navigator imports the shipper's tab bar, and tab state is local component state that resets on remount.
11	No document retention policy	LOW	Superseded KYC files persist in the bucket indefinitely. A policy decision, but one with data-protection implications given what the bucket holds.
12	Dead schema objects	LOW	<code>load_rate_visibility_enum</code> is referenced by no column; <code>subscription_type</code> is populated but unused by any feature.
13	No CI and no automated tests	LOW	No test runner or workflow configuration is committed. Every change is validated manually, which compounds findings 1 and 2.

16 Sources & Verification

Discovery was performed by direct inspection of the repository on 11 August 2026.

16.1 Verified against live code

FACT SET	SOURCE READ
Every dependency and version in section 03	Both <code>package.json</code> files
Mobile client construction, timeout, session options	<code>LoadKaro/lib/supabase.js</code>
Navigator structure, tab definitions, screen inventory	<code>navigation/*.js</code> , <code>screens/</code> , <code>screens/shipper/</code> , <code>screens/truckowner/</code>
Build profiles, package id, New Architecture flag	<code>app.json</code> , <code>eas.json</code>
Dashboard route and API inventory	<code>src/app/</code> directory tree
Dashboard library organization	<code>src/lib/</code> , <code>src/components/</code> trees
Brand tokens	<code>theme/theme.js</code>
Language coverage	<code>i18n/</code> — four dictionaries, ~355 keys
Environment variable contracts	<code>.env.example</code> files, root <code>README.md</code>

16.2 Sourced from in-repo documentation

Sections 07 and parts of 05, 06, 08, 09, and 10 — the schema catalogue, index list, RLS policy text, API parameter tables, and audit action list — are drawn from `LoadKaro/DOCS_SUPABASE_DATABASE.md` and `LoadKaro/DOCS_SYSTEM_ARCHITECTURE.md`. Those documents record a live introspection dated **8 April 2026**.

VERIFICATION GAP

A live re-introspection of the database was attempted for this document and could not be completed — the Supabase management connection returned an authorization error. Two RLS migrations are known to have been applied after the April introspection (snapshots dated **25 May 2026** and **1 July 2026** sit in the repository root), so the policy text in section 10 should be treated as *indicative of the model* rather than a byte-exact copy of the policies now in force. Table structure, foreign keys, enums, and constraints are unlikely to have changed, since those migrations targeted policies. Re-running introspection with a valid Supabase access token would close this gap.

16.3 Companion documents in this repository

DOCUMENT	COVERS
<code>LoadKaro/DOCS_SYSTEM_ARCHITECTURE.md</code>	Flow diagrams, API reference, role matrix
<code>LoadKaro/DOCS_SUPABASE_DATABASE.md</code>	Full schema, indexes, RLS policy text
<code>LoadKaro/DOCS_MOBILE_APP.md</code>	Screen-by-screen mobile reference
<code>LoadKaro/DOCS_ADMIN_DASHBOARD.md</code>	Dashboard component and page reference
<code>reports/02_SECURITY_AND_COMPLETENESS_AUDIT.md</code>	Prior security audit
<code>reports/06_BLOCKERS_AND_PREWARNINGS.md</code>	Launch blockers
<code>QA_IMPLEMENTATION_HISTORY.md</code>	QA defect history

LoadKaro Platform Discovery Document · Revision 1.0 · 11 August 2026 · Generated from repository inspection. Section 16.2 records the one area where facts were inherited rather than re-verified.